

# Homework 03

## Question 1: Build an explain-to-me skill

### Answer

I created a short skill named `explain-to-me` and saved it in `explain-to-me/SKILL.md`. I also installed the skill in the workspace skill directory at `.github/skills/explain-to-me/SKILL.md`.

To test the skill, I explicitly asked the agent to use `explain-to-me` to read Question 2. The skill explained that Question 2 asks me to study ridge regression when two predictors are nearly collinear. It clarified that I need to generate one fixed standardized design matrix, repeatedly generate new training and test responses, compare OLS with ridge penalties, and interpret the results using bias and variance. This was useful because it explained the task without solving the problem for me.

## Question 2: Ridge regression with nearly collinear predictors

### Answer to (a), (b), and (c)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(43231)

n = 100
p = 4
beta = np.array([1.5, 1.5, 1.0, 0.0])
lambdas = [0, 0.02, 0.2]
repetitions = 200
```

```

L = rng.standard_normal(n)
U1 = rng.standard_normal(n)
U2 = rng.standard_normal(n)
X3 = rng.standard_normal(n)
X4 = rng.standard_normal(n)

X_raw = np.column_stack([
    L + 0.04 * U1,
    L + 0.04 * U2,
    X3,
    X4
])

x_bar = X_raw.mean(axis=0)
X_centered = X_raw - x_bar
x_scale = np.sqrt(np.mean(X_centered**2, axis=0))
X = X_centered / x_scale

mu = X @ beta

cor_x1_x2 = np.corrcoef(X[:, 0], X[:, 1])[0, 1]
singular_values = np.linalg.svd(X, compute_uv=False)

print("Correlation between X1 and X2:", round(cor_x1_x2, 4))
print("Largest singular value:", round(singular_values.max(), 3))
print("Smallest singular value:", round(singular_values.min(), 3))

gram = X.T @ X
results = []

for k in range(repetitions):
    epsilon = rng.standard_normal(n)
    epsilon_test = rng.standard_normal(n)

    y = mu + epsilon
    y_test = mu + epsilon_test

    y_bar = y.mean()
    y_centered = y - y_bar

    for lam in lambdas:
        beta_hat = np.linalg.solve(

```

```

        gram + n * lam * np.eye(p),
        X.T @ y_centered
    )

    y_hat = y_bar + X @ beta_hat

    train_mse = np.mean((y - y_hat)**2)
    test_mse = np.mean((y_test - y_hat)**2)

    results.append({
        "lambda": lam,
        "beta1": beta_hat[0],
        "beta2": beta_hat[1],
        "beta_sum": beta_hat[0] + beta_hat[1],
        "beta_difference": beta_hat[0] - beta_hat[1],
        "train_mse": train_mse,
        "test_mse": test_mse
    })

results = pd.DataFrame(results)

summary = results.groupby("lambda").agg(
    beta1_mean=("beta1", "mean"),
    beta1_sd=("beta1", "std"),
    beta2_mean=("beta2", "mean"),
    beta2_sd=("beta2", "std"),
    beta_sum_mean=("beta_sum", "mean"),
    beta_sum_sd=("beta_sum", "std"),
    beta_difference_mean=("beta_difference", "mean"),
    beta_difference_sd=("beta_difference", "std"),
    average_train_mse=("train_mse", "mean"),
    average_test_mse=("test_mse", "mean")
).reset_index()

print(summary.round(3).to_string(index=False))

```

Correlation between X1 and X2: 0.9983

Largest singular value: 14.173

Smallest singular value: 0.406

| lambda | beta1_mean | beta1_sd | beta2_mean | beta2_sd | beta_sum_mean | beta_sum_sd | beta_difference_mean | beta_difference_sd |
|--------|------------|----------|------------|----------|---------------|-------------|----------------------|--------------------|
| 0.00   | 1.290      | 1.795    | 1.701      | 1.798    | 2.992         | 0.105       | 0.000                | 0.000              |
| 0.411  |            | 3.592    | 0.949      |          | 1.038         |             |                      |                    |

|       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|
| 0.02  | 1.465 | 0.145 | 1.497 | 0.148 | 2.961 | 0.104 |
| 0.032 |       | 0.274 | 0.960 |       | 1.029 |       |
| 0.20  | 1.355 | 0.049 | 1.359 | 0.050 | 2.715 | 0.095 |
| 0.004 |       | 0.029 | 1.064 |       | 1.125 |       |

The correlation between  $X_1$  and  $X_2$  is about 0.9983. The largest singular value is about 14.173, and the smallest singular value is about 0.406. This shows that the first two predictors are nearly collinear and that the design has a weak direction.

Because  $X_1$  and  $X_2$  are almost identical, the data estimate their combined effect much better than how the effect is divided between them. Therefore,  $\hat{\beta}_1 - \hat{\beta}_2$  is much more variable than  $\hat{\beta}_1 + \hat{\beta}_2$ .

Across 200 repetitions, the results were as follows.

For  $\lambda = 0$ , the empirical means were  $\hat{\beta}_1 = 1.290$ ,  $\hat{\beta}_2 = 1.701$ ,  $\hat{\beta}_1 + \hat{\beta}_2 = 2.992$ , and  $\hat{\beta}_1 - \hat{\beta}_2 = -0.411$ . The corresponding standard deviations were 1.795, 1.798, 0.105, and 3.592. The average training MSE was 0.949, and the average test MSE was 1.038.

For  $\lambda = 0.02$ , the empirical means were  $\hat{\beta}_1 = 1.465$ ,  $\hat{\beta}_2 = 1.497$ ,  $\hat{\beta}_1 + \hat{\beta}_2 = 2.961$ , and  $\hat{\beta}_1 - \hat{\beta}_2 = -0.032$ . The corresponding standard deviations were 0.145, 0.148, 0.104, and 0.274. The average training MSE was 0.960, and the average test MSE was 1.029.

For  $\lambda = 0.2$ , the empirical means were  $\hat{\beta}_1 = 1.355$ ,  $\hat{\beta}_2 = 1.359$ ,  $\hat{\beta}_1 + \hat{\beta}_2 = 2.715$ , and  $\hat{\beta}_1 - \hat{\beta}_2 = -0.004$ . The corresponding standard deviations were 0.049, 0.050, 0.095, and 0.029. The average training MSE was 1.064, and the average test MSE was 1.125.

OLS has very unstable individual coefficients because of near collinearity. Ridge greatly reduces their variance, especially for  $\hat{\beta}_1 - \hat{\beta}_2$ .

For  $\lambda = 0.02$ , the reduction in variance is larger than the increase in squared bias, so the average test MSE decreases slightly. For  $\lambda = 0.2$ , shrinkage is stronger and the bias becomes larger, so test MSE increases.

Ridge can change the individual coefficients substantially without changing fitted values as much because the combined effect  $\hat{\beta}_1 + \hat{\beta}_2$  remains relatively stable.

### Question 3: What does ridge regression shrink?

#### Answer to (a)

The ridge objective is

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{y} - X\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2.$$

Taking the gradient with respect to  $\beta$  gives

$$\nabla L_\lambda(\beta) = -\frac{1}{n}X^T(\tilde{y} - X\beta) + \lambda\beta.$$

At the minimizer, the gradient is zero, so

$$\frac{1}{n}X^T X \hat{\beta}_\lambda + \lambda \hat{\beta}_\lambda = \frac{1}{n}X^T \tilde{y}.$$

Multiplying both sides by  $n$  gives

$$(X^T X + n\lambda I_p) \hat{\beta}_\lambda = X^T \tilde{y}.$$

For any nonzero vector  $a$ ,

$$a^T (X^T X + n\lambda I_p) a = \|Xa\|_2^2 + n\lambda \|a\|_2^2.$$

When  $\lambda > 0$ , this quantity is positive. Therefore,  $X^T X + n\lambda I_p$  is positive definite and invertible, so the solution is unique even if  $X$  is rank deficient or  $p > n$ .

### **Answer to (b)**

Using the compact singular value decomposition

$$X = U_r D_r V_r^T,$$

the singular-vector directions of  $X^T X$  have eigenvalues  $d_j^2$ . Ridge adds  $n\lambda$  to each of these values, so the fitted response component in the  $j$ th left singular-vector direction is multiplied by

$$\rho_j(\lambda) = \frac{d_j^2}{d_j^2 + n\lambda}.$$

Here  $n = 100$  and  $\lambda = 0.01$ , so  $n\lambda = 1$ . The three shrinkage factors are

$$\rho_1 = \frac{10^2}{10^2 + 1} = \frac{100}{101} \approx 0.9901,$$

$$\rho_2 = \frac{2^2}{2^2 + 1} = \frac{4}{5} = 0.8,$$

and

$$\rho_3 = \frac{0.2^2}{0.2^2 + 1} = \frac{0.04}{1.04} \approx 0.0385.$$

Including the unpenalized intercept, the effective degrees of freedom are

$$df_{\text{eff}}(0.01) = 1 + 0.9901 + 0.8 + 0.0385 \approx 2.8286.$$

### **Answer to (c)**

The singular value corresponding to the third direction is the smallest. Consequently, it undergoes the strongest shrinkage effect.

Smaller singular values represent weak directions in the data. These are directions along which ordinary least squares can significantly amplify noise. Ridge regression applies shrinkage to these directions, thereby reducing prediction variance.

However, if a significant portion of the true signal lies along such a weak direction, strong shrinkage may remove part of that signal and lead to substantial prediction bias.

## **Question 4: Ridge regression and optimization**

### **Answer to (a)**

The condition number of  $A_0$  is

$$\kappa(A_0) = \frac{9}{0.01} = 900.$$

With  $\lambda = 0.09$ , the eigenvalues become

$$9.09 \quad \text{and} \quad 0.10.$$

Therefore,

$$\kappa(A_\lambda) = \frac{9.09}{0.10} = 90.9.$$

Ridge reduces the condition number, so the objective becomes less elongated and better conditioned.

**Answer to (b)**

In an eigendirection with unpenalized eigenvalue  $a$ , the error component is multiplied by

$$1 - \eta(a + \lambda).$$

For convergence in every direction,

$$|1 - \eta(a + \lambda)| < 1.$$

Using the largest eigenvalue  $M = 9.09$ ,

$$0 < \eta < \frac{2}{9.09},$$

or approximately

$$0 < \eta < 0.220.$$

For

$$\eta = \frac{1}{M},$$

the contraction factor in the largest-eigenvalue direction is

$$1 - \frac{9.09}{9.09} = 0,$$

while in the smaller direction it is

$$1 - \frac{0.10}{9.09} \approx 0.989.$$

Therefore, the high-curvature direction converges much faster, while the low-curvature direction converges slowly.

For

$$\eta = \frac{2.1}{M},$$

the contraction factor in the largest-eigenvalue direction is

$$1 - 2.1 = -1.1.$$

Since

$$|-1.1| > 1,$$

the error grows rather than contracts, so gradient descent does not converge.

### Answer to (c)

A larger  $\lambda$  can improve the condition number and make optimization faster, but this does not mean it is the best statistical choice.

Increasing  $\lambda$  also increases coefficient shrinkage and prediction bias. Therefore,  $\lambda$  should be selected based on prediction performance, such as cross-validation, rather than optimization speed.

### Question 5: Selecting the ridge penalty for real-estate prediction

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

data_path = Path("data")
if not (data_path / "realestate.csv").exists():
    data_path = Path("homework/week-03/data")

realestate = pd.read_csv(data_path / "realestate.csv")
fold_info = pd.read_csv(data_path / "realestate-split-folds.csv")

df = fold_info.merge(realestate, on="row_id", how="left")

features = ["date", "age", "distance", "stores", "latitude", "longitude"]
train = df[df["split"] == "train"]
test = df[df["split"] == "test"]

X_train_raw = train[features].to_numpy(float)
y_train = train["price"].to_numpy(float)
```

```

X_test_raw = test[features].to_numpy(float)
y_test = test["price"].to_numpy(float)
fold_id = train["cv_fold"].to_numpy(int)

lambdas = np.r_[0, 10 ** (-4 + 0.05 * np.arange(121))]

def scale_data(X_fit, X_new=None):
    center = X_fit.mean(axis=0)
    scale = np.sqrt(np.mean((X_fit - center) ** 2, axis=0))
    X_fit_scaled = (X_fit - center) / scale
    if X_new is None:
        return X_fit_scaled
    X_new_scaled = (X_new - center) / scale
    return X_fit_scaled, X_new_scaled

def ridge_fit(X, y, lam):
    n, p = X.shape
    y_bar = y.mean()
    y_centered = y - y_bar
    beta_hat = np.linalg.solve(
        X.T @ X + n * lam * np.eye(p),
        X.T @ y_centered
    )
    return y_bar, beta_hat

cv_errors = np.zeros((10, len(lambdas)))

for i, fold in enumerate(sorted(np.unique(fold_id))):
    valid = fold_id == fold
    X_fit, X_valid = scale_data(X_train_raw[~valid], X_train_raw[valid])
    y_fit = y_train[~valid]
    y_valid = y_train[valid]

    for j, lam in enumerate(lambdas):
        y_bar, beta_hat = ridge_fit(X_fit, y_fit, lam)
        y_pred = y_bar + X_valid @ beta_hat
        cv_errors[i, j] = np.mean((y_valid - y_pred) ** 2)

cv_mean = cv_errors.mean(axis=0)
cv_se = cv_errors.std(axis=0, ddof=1) / np.sqrt(10)

min_index = np.argmin(cv_mean)

```

```

lambda_min = lambdas[min_index]
one_se_limit = cv_mean[min_index] + cv_se[min_index]
lambda_1se = lambdas[cv_mean <= one_se_limit].max()

X_train = scale_data(X_train_raw)
d = np.linalg.svd(X_train, compute_uv=False)
gcv = []

for lam in lambdas:
    y_bar, beta_hat = ridge_fit(X_train, y_train, lam)
    y_hat = y_bar + X_train @ beta_hat
    df_eff = 1 + np.sum(d**2 / (d**2 + len(y_train) * lam))
    train_mse = np.mean((y_train - y_hat) ** 2)
    gcv.append(train_mse / (1 - df_eff / len(y_train)) ** 2)

gcv = np.array(gcv)
lambda_gcv = lambdas[np.argmin(gcv)]

print("lambda_min =", round(lambda_min, 5))
print("minimum CV MSE =", round(cv_mean[min_index], 2))
print("lambda_1se =", round(lambda_1se, 5))
print("lambda_GCV =", round(lambda_gcv, 5))

X_train, X_test = scale_data(X_train_raw, X_test_raw)
rows = []

for model, lam in [
    ("OLS", 0),
    ("lambda_min", lambda_min),
    ("lambda_1se", lambda_1se),
    ("lambda_GCV", lambda_gcv),
]:
    y_bar, beta_hat = ridge_fit(X_train, y_train, lam)
    train_pred = y_bar + X_train @ beta_hat
    test_pred = y_bar + X_test @ beta_hat
    df_eff = 1 + np.sum(d**2 / (d**2 + len(y_train) * lam))
    rows.append({
        "model": model,
        "lambda": lam,
        "train_mse": np.mean((y_train - train_pred) ** 2),
        "test_mse": np.mean((y_test - test_pred) ** 2),
        "df_eff": df_eff,
    })

```

```

})

final_table = pd.DataFrame(rows)
print(final_table.round(3).to_string(index=False))

positive = lambdas > 0
fig, axes = plt.subplots(1, 2, figsize=(8, 3.5))

axes[0].plot(np.log10(lambdas[positive]), cv_mean[positive])
axes[0].axvline(np.log10(lambda_min), linestyle="--")
axes[0].axvline(np.log10(lambda_1se), linestyle=":")
axes[0].set_xlabel("log10(lambda)")
axes[0].set_ylabel("CV MSE")

axes[1].plot(np.log10(lambdas[positive]), gcv[positive])
axes[1].axvline(np.log10(lambda_gcv), linestyle="--")
axes[1].set_xlabel("log10(lambda)")
axes[1].set_ylabel("GCV")

fig.tight_layout()
plt.show()

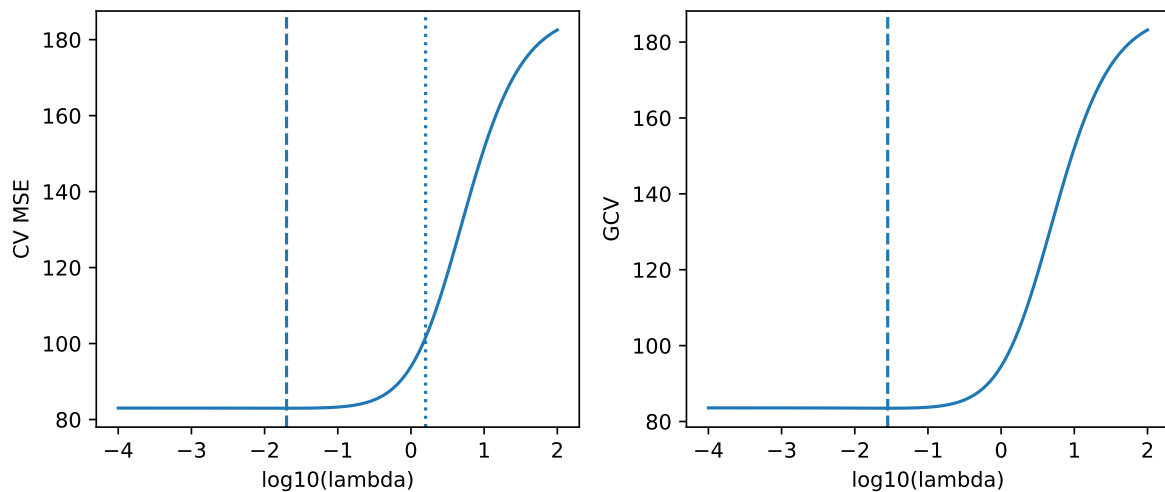
```

```

lambda_min = 0.01995
minimum CV MSE = 82.98
lambda_1se = 1.58489
lambda_GCV = 0.02818

```

|            | model | lambda | train_mse | test_mse | df_eff |
|------------|-------|--------|-----------|----------|--------|
|            | OLS   | 0.000  | 80.094    | 66.322   | 7.000  |
| lambda_min |       | 0.020  | 80.135    | 66.000   | 6.773  |
| lambda_1se |       | 1.585  | 100.374   | 81.200   | 3.033  |
| lambda_GCV |       | 0.028  | 80.171    | 65.903   | 6.689  |



### Answer to (a)

Using the supplied ten training folds and estimating predictor means and scales within each set of nine training folds, the minimum cross-validation error occurred at

$$\lambda_{\min} \approx 0.01995.$$

The mean validation MSE at this value was approximately

$$82.98.$$

Using the one-standard-error rule, the largest candidate within one standard error of the minimum was

$$\lambda_{1se} \approx 1.58489.$$

Thus, the one-standard-error rule selects a much stronger penalty than the minimum-error rule.

### Answer to (b)

Using all 332 training observations, the GCV criterion was minimized at

$$\lambda_{GCV} \approx 0.02818.$$

This value is close to  $\lambda_{\min}$ , so cross-validation and GCV give similar penalty choices in this dataset.

### Answer to (c)

After fixing all three penalty choices, the models were refitted using all training observations and then evaluated on the final test set.

| Model            | $\lambda$ | Train MSE | Test MSE | Effective df |
|------------------|-----------|-----------|----------|--------------|
| OLS              | 0         | 80.094    | 66.322   | 7.000        |
| $\lambda_{\min}$ | 0.01995   | 80.135    | 66.000   | 6.773        |
| $\lambda_{1se}$  | 1.58489   | 100.374   | 81.200   | 3.033        |
| $\lambda_{GCV}$  | 0.02818   | 80.171    | 65.903   | 6.689        |

The minimum-error CV and GCV models use only slightly more regularization than OLS, so their training and test MSE values are similar. The one-standard-error rule uses a much stronger penalty, which lowers the effective degrees of freedom but increases both training and test errors.

Although  $\lambda_{GCV}$  has the smallest observed test MSE in the table, it should not be selected after looking at the test results. The test set must stay independent of model selection, otherwise the reported test performance will be optimistically biased.

### explain-to-me skill

```
---
name: explain-to-me
description: Explain what a homework question is asking without solving it.
---
```

```
# Explain to Me
```

```
When I explicitly ask you to use this skill on a homework question:
```

1. Explain the main goal of the question in simple language.
2. Break the question into the specific tasks I need to complete.
3. Identify the statistical or computational concepts I need to understand.
4. Point me to the most relevant section of the Week 3 lectures:
  - Ridge Regression: Stability Through Shrinkage
  - From a Penalized Objective to a Fitted Ridge Model
5. Warn me about important details or common mistakes in the question.
6. Do not solve the problem, provide final numerical answers, or write the complete code unless asked.

Keep the explanation concise and focused on helping me understand what I am supposed to do.